pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

pandas: Combining and Loading, Part 2

WEEK 2, DAY 3, PART 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's thirteenth note

Part 1's averages gave every bond one vote. Give me spread and duration weighted by market value, by sector too, and tell me which sector moves and why. Then market value by sector and grade in one grid. How much matures in the next three years? Send it as a workbook I can open, and leave nothing behind.

```
holdings = pd.read_csv(data_folder + 'holdings.csv')
ratings = pd.read_csv(data_folder + 'ratings.csv')
book = pd.merge(holdings, ratings, on='rating')
sector_summary = book.groupby('sector').agg(
    bonds=('cusip', 'count'),
    market_value=('market_value', 'sum'),
    avg_oas=('oas', 'mean'),
    avg_duration=('duration', 'mean'),
)
sector_summary = sector_summary.round(2)
print(book.shape, sector_summary.shape)
(200, 22) (10, 4)
```

A new notebook starts empty. The issuers are invented.

Same 200 bonds, same "book": every figure checks against Part 1.

8.7 One vote per dollar: the book, then each sector

```
oas_x_value = book['oas'] * book['market_value']
weighted_oas = oas_x_value.sum() / book['market_value'].sum()
print('Simple:', round(book['oas'].mean(), 2), 'Weighted:', round(weighted_oas, 2))
Simple: 185.75 Weighted: 180.46
```

```
duration_x_value = book['duration'] * book['market_value']
weighted_duration = duration_x_value.sum() / book['market_value'].sum()
print('Simple:', round(book['duration'].mean(), 2), 'Weighted:', round(weighted_duration, 2))
Simple: 6.47 Weighted: 6.25
```

```
book['oas_x_value'] = book['oas'] * book['market_value']
sector_sums = book.groupby('sector')[['oas_x_value', 'market_value']].sum()
weighted_by_sector = sector_sums['oas_x_value'] / sector_sums['market_value']
by_sector = pd.DataFrame({'simple': book.groupby('sector')['oas'].mean(),
                          'weighted': weighted_by_sector})
by_sector['gap'] = by_sector['weighted'] - by_sector['simple']
by_sector.round(1)
```

```
weighted_oas_column = weighted_by_sector.rename('weighted_oas').round(2)
sector_summary = sector_summary.join(weighted_oas_column)
```

Weight by market value, and spread and duration both come in lower.

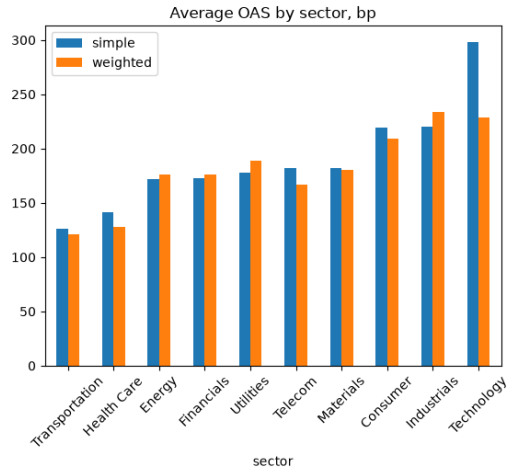
sector	simple	weighted	gap
Consumer	218.9	209.4	-9.5
Energy	171.6	176.3	4.8
Financials	172.3	176.1	3.8
Health Care	141.8	128.0	-13.7
Industrials	220.6	234.0	13.4
Materials	182.4	180.7	-1.7
Technology	298.2	228.2	-69.9
Telecom	182.4	166.9	-15.5
Transportation	126.5	120.9	-5.6
Utilities	177.9	188.8	10.9

The two totals divide sector by sector because both carry the sector as label, as in pandas Part 1.

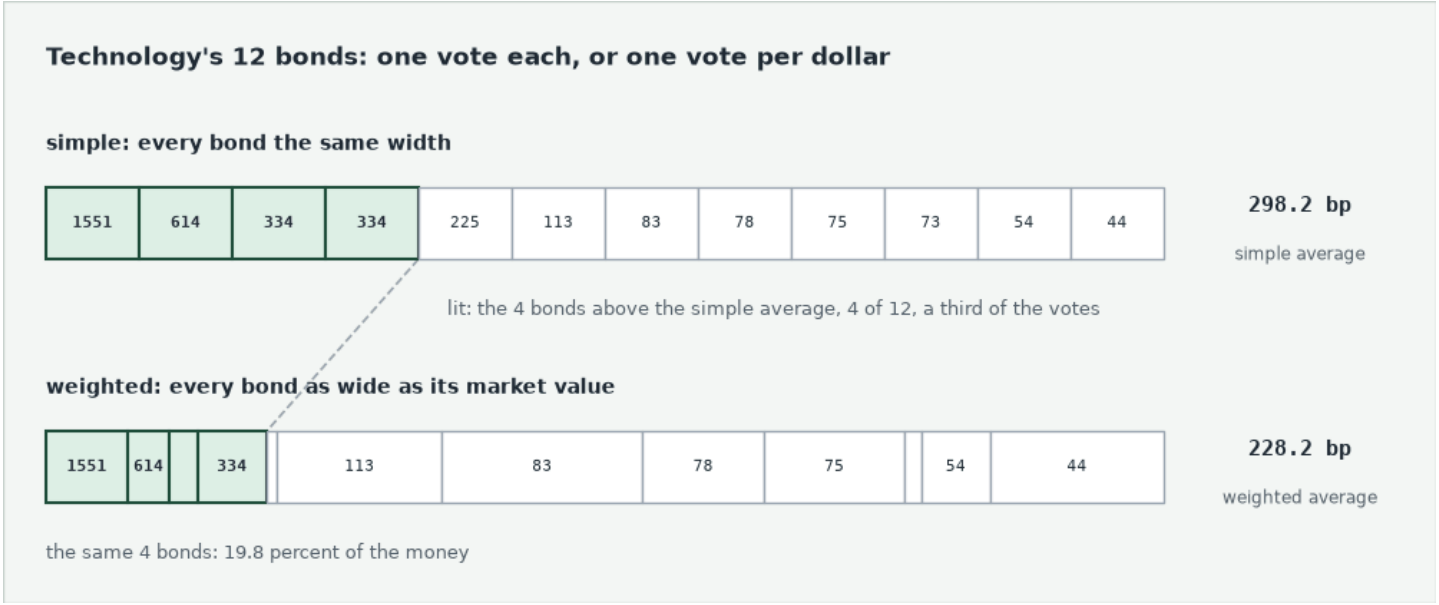
Excel side by side

NumPy Part 1's =SUMPRODUCT(M2:M201, S2:S201) / SUM(S2:S201) weights the whole book, OAS in M and market value in S. One sector: =SUMPRODUCT((E2:E201="Energy") * M2:M201 * S2:S201) / SUMIFS(S2:S201, E2:E201, "Energy"), sector in E.

Simple beside weighted, and why Technology moves



The weighted figure is lower in six sectors and higher in four: Energy, Financials, Industrials, and Utilities.



Close in nine sectors; Technology falls 70 bp, from 298.2 to 228.2.

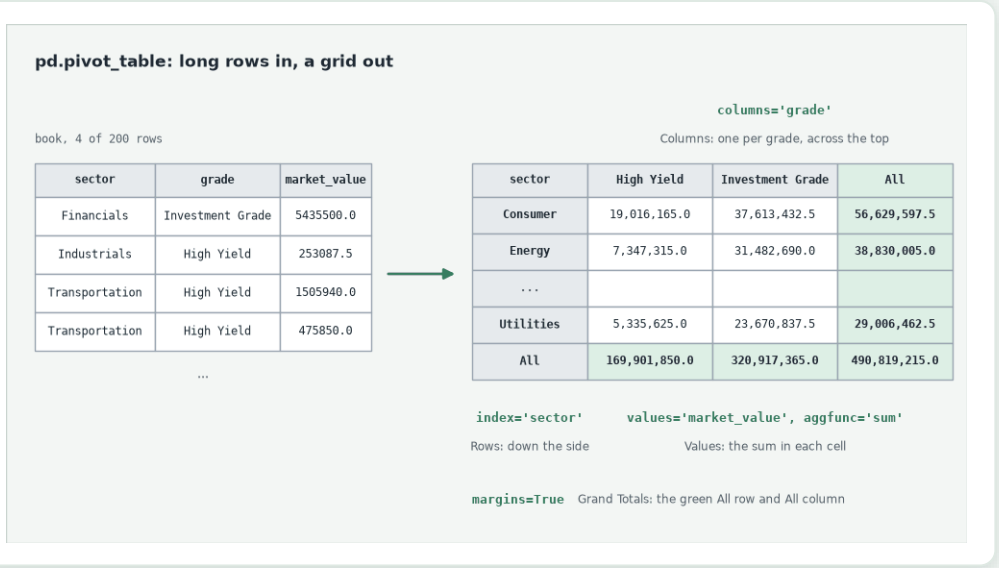
```
technology = book[book['sector'] == 'Technology']
above = technology[technology['oas'] > technology['oas'].mean()]
share = above['market_value'].sum() / technology['market_value'].sum() * 100
print('Bonds:', above.shape[0], 'of', technology.shape[0])
print('Share of value:', round(share, 1))
Bonds: 4 of 12
Share of value: 19.8
```

Neither is wrong: a typical bond against a typical dollar.

8.8 Sector down, grade across, totals included

```
pivot = pd.pivot_table(book, index='sector', columns='grade', values='market_value',
                       aggfunc='sum', margins=True)
pivot.style.format('{:,.1f}').set_caption('Market value by sector and grade')
```

sector	High Yield	Investment Grade	All
Consumer	19,016,165.0	37,613,432.5	56,629,597.5
Energy	7,347,315.0	31,482,690.0	38,830,005.0
Financials	15,335,395.0	52,884,277.5	68,219,672.5
Health Care	12,891,097.5	26,017,090.0	38,908,187.5
Industrials	22,802,335.0	29,738,410.0	52,540,745.0
Materials	38,747,115.0	32,954,502.5	71,701,617.5
Technology	6,258,692.5	24,053,067.5	30,311,760.0
Telecom	27,796,585.0	28,051,685.0	55,848,270.0
Transportation	14,371,525.0	34,451,372.5	48,822,897.5
Utilities	5,335,625.0	23,670,837.5	29,006,462.5
All	169,901,850.0	320,917,365.0	490,819,215.0



The corner, 490,819,215, is the book's total from pandas Part 2. Materials is the one sector with more in high yield, 38,747,115, than investment grade, 32,954,502.5; in Telecom the two are near equal.

One line, one PivotTable, totals included.

A label from a rule: apply, lambda, np.where

```
def price_label(price):
    if price > 100:
        return 'Premium'
    elif price < 100:
        return 'Discount'
    else:
        return 'Par'
```

```
book['size_label'] = book['par_held'].apply(
    lambda face: '1mm and over' if face >= 1000000 else 'Under 1mm')
book['size_label'].value_counts()
size_label
1mm and over    171
Under 1mm        29
Name: count, dtype: int64
```

```
book['price_label'] = book['price'].apply(price_label)
book['price_label'].value_counts()
price_label
Discount    113
Premium      87
Name: count, dtype: int64
```

```
import numpy as np
by_column = np.where(book['price'] > 100, 'Premium',
                     np.where(book['price'] < 100, 'Discount', 'Par'))
agree = (by_column == book['price_label']).sum()
print(agree, 'of', book.shape[0], 'rows agree')
assert agree == book.shape[0], 'the two labels differ'
200 of 200 rows agree
```

Excel side by side. Price in column I

Job	pandas	Excel
Sector by grade	`pd.pivot_table(book, index='sector', columns='grade', values='market_value', aggfunc='sum')`	a PivotTable:
		sector in Rows, grade in Columns, Sum of market_value in Values
Totals	`margins=True`	Grand Totals, on by default
A label from a rule	`apply(price_label)`, or the nested `np.where`	=IF(I2>100, "Premium", IF(I2<100, "Discount", "Par")) filled down

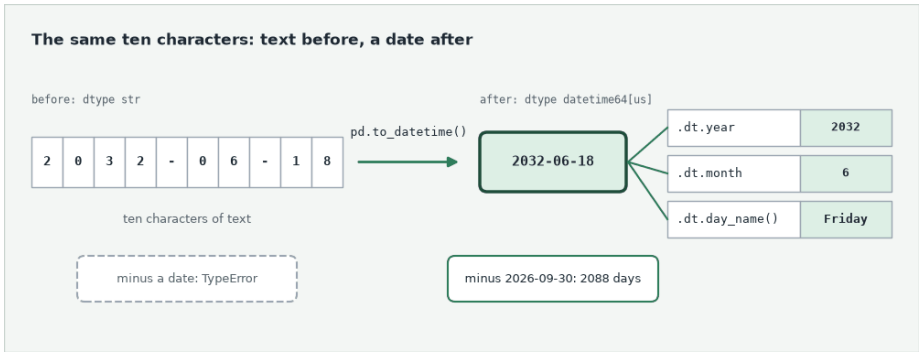
Column tools first; "apply" for a rule they cannot say.

8.9 Maturity is text until it is converted

```
book['maturity'].head(3)
0    2032-06-18
1    2029-04-17
2    2030-06-03
Name: maturity, dtype: str
```

```
as_of = pd.Timestamp('2026-09-30')
book['maturity'] - as_of
TypeError: operation 'sub' not supported for dtype 'str' with object of type <class 'pandas.Timestamp'>
```

```
book['maturity'] = pd.to_datetime(book['maturity'])
book['maturity'].head(3)
0   2032-06-18
1   2029-04-17
2   2030-06-03
Name: maturity, dtype: datetime64[us]
```



Text cannot be subtracted: convert it first.

The same values, now "datetime64[us]": real dates, kept to the microsecond (older pandas shows "[ns]", the nanosecond). Text in another layout, such as 06/18/2032, needs the layout named: "pd.to_datetime(column, format='%m/%d/%Y')", with "%m" the month, "%d" the day, and "%Y" the four-digit year.

Year, month, weekday, and years to maturity

```
book['maturity_year'] = book['maturity'].dt.year
book['maturity_month'] = book['maturity'].dt.month
book['maturity_day'] = book['maturity'].dt.day_name()
book[['ticker', 'maturity', 'maturity_year', 'maturity_month', 'maturity_day']].head(3)
```

	ticker	maturity	maturity_year	maturity_month	maturity_day
0	BEZE	2032-06-18	2032	6	Friday
1	BEZH	2029-04-17	2029	4	Tuesday
2	BEZL	2030-06-03	2030	6	Monday

```
days_left = (book['maturity'] - as_of).dt.days
book['years_to_maturity'] = days_left / 365.25
book['years_to_maturity'].agg(['min', 'max', 'mean']).round(2)
min      1.00
max     29.90
mean     9.85
Name: years_to_maturity, dtype: float64
```

Excel stores a date as a count of days, so a subtraction gives days there too.

Excel side by side. Report date in A, maturity in H, as in holdings.csv

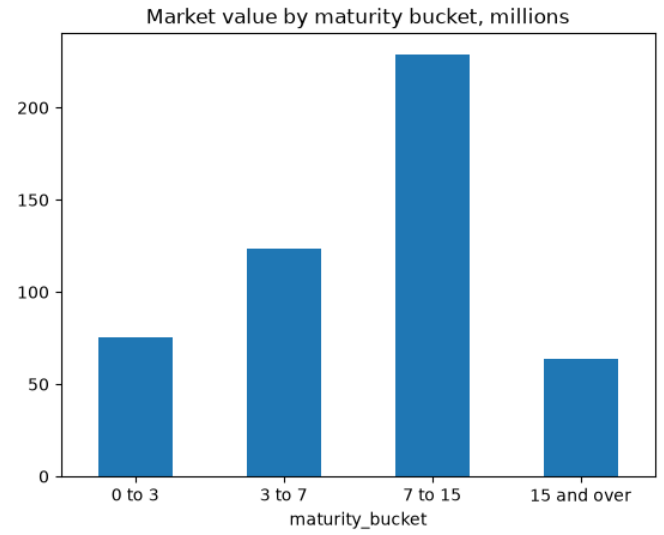
Job	pandas	Excel
Text to a date	`pd.to_datetime(book['maturity'])`	=DATEVALUE(H2), for a date that arrives as text
Year, month, weekday	`.dt.year, .dt.month, .dt.day_name()`	=YEAR(H2), =MONTH(H2), =TEXT(H2, "dddd")
Day first	`dayfirst=True`	Data, Text to Columns, column data format Date, DMY
A date as text	`.dt.strftime('%d %b %Y')`	=TEXT(H2, "dd mmm yyyy")
Years left	`(book['maturity'] - as_of).dt.days / 365.25`	=(H2 - A2) / 365.25
Three years on	`pd.Timestamp('2029-09-30')`	=EDATE(A2, 36)
Bonds before 2028	`book['maturity'] < pd.Timestamp('2028-01-01')`	=COUNTIF(H:H, "<" & DATE(2028, 1, 1))
Maturity bucket	`pd.cut(...)`	nested IF, or VLOOKUP approximate on a table of edges

Maturity buckets and the maturity profile

```
edges = [0, 3, 7, 15, 100]
names = ['0 to 3', '3 to 7', '7 to 15', '15 and over']
book['maturity_bucket'] = pd.cut(book['years_to_maturity'], bins=edges, labels=names)
book['maturity_bucket'].value_counts().sort_index()
maturity_bucket
0 to 3         28
3 to 7         51
7 to 15        88
15 and over    33
Name: count, dtype: int64
```




```
within_three = book['maturity'] <= pd.Timestamp('2029-09-30')
print('Bonds:', within_three.sum())
print('Market value:', book.loc[within_three, 'market_value'].sum())
Bonds: 28
Market value: 75125702.5
```



28 bonds and 75.1 million inside three years; the most money 7 to 15 years out.

8.10 Save the book, read it back, keep the sector names

```
book.to_csv('desk_book_with_grades.csv')
read_back = pd.read_csv('desk_book_with_grades.csv')
print('Saved:', book.shape, 'Read back:', read_back.shape)
Saved: (200, 30) Read back: (200, 31)
```

	Unnamed: 0	as_of_date	cusip
0	0	2026-09-30	99080CAE8
1	1	2026-09-30	99049XAE2
2	2	2026-09-30	99039NAC0

The extra column, "Unnamed: 0", holds the row labels, saved with no header.

```
book.to_csv('desk_book_with_grades.csv', index=False)
read_back = pd.read_csv('desk_book_with_grades.csv')
print('Saved:', book.shape, 'Read back:', read_back.shape)
Saved: (200, 30) Read back: (200, 30)
```

```
sector_summary.to_csv('sector_summary.csv')
pd.read_csv('sector_summary.csv').head(3)
```

	sector	bonds	market_value	avg_oas	avg_duration	weighted_oas
0	Consumer	26	56629597.5	218.88	6.84	209.43
1	Energy	17	38830005.0	171.59	6.97	176.35
2	Financials	26	68219672.5	172.31	6.15	176.08

A grouped table keeps its index: without it, no row says which sector it is.

Excel side by side. "to_csv" is File, Save As, CSV; "read_csv" is File, Open.

8.11 The book as a workbook, then two sheets in one

```
book.to_excel('desk_report.xlsx', sheet_name='book', index=False)
assert from_excel.shape == book.shape, 'the sheet is not the book'
assert np.isclose(from_excel['market_value'].sum(), book['market_value'].sum())
print(from_excel.shape)
(200, 30)
```

```
with pd.ExcelWriter('desk_report.xlsx') as writer:
    book.to_excel(writer, sheet_name='book', index=False)
    sector_summary.to_excel(writer, sheet_name='by_sector')
all_sheets = pd.read_excel('desk_report.xlsx', sheet_name=None)
for name, table in all_sheets.items():
    print(name, table.shape)
os.remove('desk_report.xlsx')
print(os.path.exists('desk_report.xlsx'))
book (200, 30)
by_sector (10, 6)
False
```

	saved	read back
cusip	str	str
maturity	datetime64[us]	datetime64[us]
maturity_bucket	category	str

Formulas never travel: as in pandas Part 2, a column holds values.

A few lines write the report; next month they run again.

Excel side by side. What pandas writes is what Paste Special, Values leaves.

Job	pandas	Excel
Save a workbook	`book.to_excel('desk_report.xlsx', sheet_name='book', index=False)`	File, Save As, Excel Workbook
Open one sheet	`pd.read_excel('desk_report.xlsx', sheet_name='book')`	File, Open, then the sheet's tab
Two tables, two sheets	`pd.ExcelWriter, one to_excel each`	two sheets, each with its tab

Predict first: decide, then run each in the notebook

Predict 1

The next cell converts "03/04/2029" again, without "dayfirst". Which date comes back? A) The 3rd of April, as above. B) The 4th of March. C) An error: the date reads two ways.

```
print(pd.to_datetime(pd.Series(['03/04/2029']))[0])
```

Predict 2

Another system's file holds two invented CUSIPs made only of digits, 009900123 and 099011457. What does "read_csv" show for the first? A) 009900123. B) 9900123. C) An error.

```
typed_file = '''cusip,par_held
009900123,1000000
099011457,2500000
'''
import io
as_numbers = pd.read_csv(io.StringIO(typed_file))
as_numbers['cusip']
```

Predict 3

The book was saved with "maturity" as dates. The next cell reads the file again with "parse_dates=['maturity']" and shows the types of three date columns, "as_of_date", "maturity", and "price_date", beside the plain read's. With "parse_dates=", how many of the three come back as dates? Write down a number.

```
reloaded = pd.read_csv('desk_book_with_grades.csv', parse_dates=['maturity'])
three = ['as_of_date', 'maturity', 'price_date']
pd.DataFrame({'plain': read_back[three].dtypes, 'parse_dates': reloaded[three].dtypes})
```

The note, answered

The note asked	Answer
Spread and duration weighted by market value	180.46 bp and 6.25 years, against 185.75 and 6.47 simple
Which sector moves, and why	Technology, 298.2 to 228.2: four wide bonds, a third of its count and 19.8 percent of its money
Market value by sector and grade	one pivot_table, totals included
What matures in three years	28 bonds, 75.1 million
A workbook, nothing left behind	two sheets, read back and checked; every file removed

Be proud: the weighted figures, the grid, and the maturity profile are a desk's report, and it runs again next month on a new file.

Every file today was the desk's own and clean. In the week's case study three files arrive from three systems and nobody says what is wrong with them: look before you trust.